TEAM DODO 기획제안서

프로젝트명: DODO JOURNEY

팀원1: 정의훈

팀원2: 채승우

팀원3: 최환희

팀원4: 한도훈

목차

1. 프로젝트 개요 4
1.1 문제정의 및 솔루션 4
1.2 주 타겟 및 Stakeholder 4
1.3 핵심 가치 5
2. 기존 사례 분석 및 사업화 가능성 6
2.1 유사 제품/서비스 6
2.1.1 당근(동네생활) 6
2.1.2 구글 맵스(장소 리뷰) 6
2.1.3 포켓몬GO 6
2.2 사업화 가능성 7
2.2.1 비즈니스 모델 정의 7
2.2.2 시장규모 분석 7
2.2.3 상용화 시 예상 사업규모 8
3. User Story Mapping 9
3.1 페르소나 정의 9
3.1.1 Persona 1 - 산책자 9
3.1.2 Persona 2 - 기록자 9
3.1.3 Persona 3 - 서비스 관리자 10
3.2 사용자 여정 (User Journey) 10
3.2.1 산책자 Journey 10
3.2.2 기록자 Journey 10
3.2.3 관리자 Journey 11
3.3 Epics & User Stories 11
3.3.1 Epic 0: 앱 실행 및 회원가입&로그인 11
3.3.2 Epic 1: 경로 탐색 및 트래킹 11
3.3.3 Epic 2: 로컬 메시지 생성 및 관리 11
3.3.4 Epic 3: 위치 기반 콘텐츠 해금 12
3.3.5 Epic 4: 인터랙션 및 마이페이지&엽서 시스템 12
3.3.6 Epic 5: 운영 및 품질 관리 12
4. 설계 및 구현 13
4.1 기술 분석 13
4.1.1 기술 스택 13
4.1.1.1 Backend Stack 14
4.1.1.2 Frontend Stack 15
4.1.1.3 Android Native 17

4.1.2 개발 및 협업 환경 19
4.2 시스템 설계 20
4.2.1 시스템 개요 20
4.2.2 소프트웨어 아키텍처 21
4.2.3 데이터 설계 23
4.2.3.1 사용자 관리 24
4.2.3.2 핵심 콘텐츠 25
4.2.3.3 위치 및 카테고리 26
4.2.3.4 운영 28
4.2.4 API 정의 29
5. 과제 수행 계획 34
5.1 일정계획 34
5.2 업무분장 35
5.3 코드 품질 향상을 위한 협업 방식 35
5.3.1 git branch 사용 방식 35
5.3.2 DevOps 파이프라인 구성 36
5.3.3 테스트 자동화 방식 37
5.3.3.1 Backend 37
5.3.3.2 Frontend 37
5.3.3.3 Android 38
5.3.4 코드 리뷰 방식 40
5.3.5 코드 컨벤션 40
5.3.5.1 Git 컨벤션 40
5.3.5.2 BE 컨벤션 41
5.3.5.3 FE 컨벤션 41
5.3.5.4 AN 컨벤션 42
5.4 성과 측정 방안 43
5.4.1 비용적 KPI 트리 43
5.4.2 기능적 KPI 목록 43
5.4.2.1 [BE] KPI 43
5.4.2.2 [FE] KPI 44
5.4.2.3 [AN] KPI 44

1. 프로젝트 개요

1.1 문제정의 및 솔루션

기존 단조로운 산책의 한계는 산책이 단순한 운동이나 이동 수단에 그치며, 산책의 과정에서 얻을 수 있는 즐거움이 부족하다는 점이다.

- 늘 다니던 길로만 향하게되는 기존 산책으로 인해 새로운 발견이 힘들다.
- 목적지 중심의 이동 외에는 즐길 거리 부족함으로 지루하게 느껴질 수 있다.

단조로운 산책을 타파하기 위해 주변 정보를 찾아보지만 광고성 정보의 범람으로 사용자의 검색 피로도가 누적된다.

- 주변 정보 검색 시 원하는 정보 대신 광고성 정보만 보이게되어 검색 피로도가 증가한다.
- 신뢰도 높은 적절한 정보를 찾기 어렵다.

이를 위한 Solution으로는 위치 기반으로 컨텐츠를 생성하고 소비하는 방안을 제안한다. *(걷는 자만이 가질 수 있는 정보)*

- 산책 중에 할 수 있는 새로운 컨텐츠 제공하여 과정의 재미 준다.
- 블러 및 프리뷰로 궁금증 유발하고 산책의 탐험화, 동기부여를 준다.
- 물리적 방문 인증 시스템으로 정보의 신뢰도 확보와 검색 피로도를 완화한다.

1.2 주 타겟 및 Stakeholder

주 타겟은 반복되는 동네 산책에 무료함을 느껴 재미요소를 찾는 산책자이다.

더 나아가 확장 타겟으로는 산책과 같은 가벼운 운동을 시작하고 싶지만 너무 무거운 일로 느껴졌던 예비 운동가와 장소를 매개로 시공간적 유대감을 쌓고 싶은 SNS 유저를 생각해 볼 수 있다.

Stakeholder

- 산책자
  - 니즈: 재미있는 이동 동기와 신뢰성 높은 정보를 원한다.
  - 제공가치: 지오캐싱 기반 탐험 컨텐츠로 동기를 부여하며 인증된 로컬 정보로 신뢰성을 보장한다.
- 기록자
  - 니즈: 장소 기반 에피소드 기록 아카이빙을 원한다.
  - 제공가치: 특정 장소에 귀속되는 메시지로 지도 위에 사용자만의 아카이브를 구축할 수 있도록 한다.
- 운영자
  - 니즈: 서비스 품질과 커뮤니티를 관리하길 원한다.
  - 제공가치: 신고 시스템 기반 콘텐츠 필터링으로 유저에게 적절한 제재를 가할 수 있으며 데이터분석을 제공해 서비스를 관리하기 용이하게 한다.
- 잠재적 파트너
  - 니즈: 방문객을 유도하고 방문 가능성이 높은 로컬 마케팅을 원한다.
  - 제공가치: 위치 기반 서비스를 통해 로컬 사용자 타겟 광고를 할 수 있도록 한다.

1.3 핵심 가치

- 색다른 컨텐츠 제공 방식과 탐험 요소로 산책에 재미를 줄 수 있다.
- 위치 기반 기록 아카이브 구축을 통해 내 공간을 확장한다.
- 익명으로 연결된 지역 커뮤니티 소속감을 제공한다.

따라서 DODO를 통해 다음과 같은 가치를 제공하고자 한다.

[탐험] - 익숙한 공간에서 예상치 못한 콘텐츠를 발견하는 즐거움 제공한다.

[연결] - 익명성에 기반해 '장소'라는 공통 분모를 통해 연결되는 새로운 형태의 로컬 커뮤니티를 형성한다.

[신뢰] - 위치 인증 기반으로 믿고 즐길 수 있는 콘텐츠 생태계를 구축한다.

2. 기존 사례 분석 및 사업화 가능성

2.1 유사 제품/서비스

2.1.1 당근(동네생활)

- 핵심 기능 - GPS를 바탕으로 물리적으로 인접한 동네 주민들끼리 지역 기반 정보, 중고 거래, 일상 질문 등을 나누는 하이퍼로컬 커뮤니티 기능이다.
- 도출 인사이트 - 유저들은 자신이 속한 지역 커뮤니티에 소속감을 느끼며, 물리적으로 가까운 위치의 사람들과 소통하고 정보를 교류하는 행위가 잠재 수요가 있음을 보여준다.
- DODO 차별화 포인트 - 당근이 보다 넓은 지역(e.g., 망포동)을 기준으로 정보 공유와 일상적인 대화등을 나누는 기능을 제공하는 반면, DODO는 상대적으로 더 좁은 장소에 얽힌 개인의 경험, 추억, 감정을 남기고 발견하는 정서적 교류에 집중하여 보다 감성적인 연결망을 구축한다.

2.1.2 구글 맵스(장소 리뷰)

- 핵심 기능 - 특정 장소(식당, 카페, 명소 등)에 대한 지도 내비게이션 기능과 함께, 실제 방문자가 별점, 사진, 텍스트 형태의 평가를 남겨 다른 사람과 공유하는 기능이다.
- 도출 인사이트 - 유저들은 자신이 방문한 오프라인 장소에 디지털 형태로 흔적을 남기려는 자발적인 기록 욕구가 있으며, 이러한 사용자 생성 콘텐츠가 타인의 행동에 큰 영향을 미친다는 점을 시사한다.
- DODO 차별화 포인트 - 구글 맵스가 주로 상업적 장소의 평가에 한정되어 있다면, DODO는 상업 시설뿐만 아니라 이름 없는 벤치, 동네 산책로, 특정 좌표 등 일상의 모든 공간을 사용자 간의 사적인 메시지 소통 창구로 확장하여 공간의 의미를 재정의한다.

2.1.3 포켓몬GO

- 핵심 기능 - 증강현실과 위치기반 서비스를 결합하여, 유저가 현실 세계의 지도를 따라 직접

이동하며 특정 좌표에서 가상의 포켓몬을 포획하거나 보상을 얻는 게임이다.

- 도출 인사이트 - 디지털 화면 속의 콘텐츠와 보상이 유저들을 실제 물리적 공간(오프라인)으로 직접 이동하게 만드는 행동 유발 효과가 있음을 증명한다.
- DODO 차별화 포인트 - 포켓몬GO가 가상의 포켓몬 수집을 목적으로 유저를 걷게 만든다면, DODO는 그 목적을 타인이 남긴 이야기와 감정의 발견으로 전환한다. 게임 시스템이나 유명 IP가 아닌, 누군가 남긴 장소에 관한 메시지를 찾아내는 아날로그적 감성과 특정 장소에 남겨진 사람의 이야기를 통한 사람 간의 연결 자체를 보상으로 제공한다.

2.2 사업화 가능성

2.2.1 비즈니스 모델 정의

DODO는 거창한 수익 구조 보다 서비스의 자생적인 운영과 지역 활성화에 초점을 맞춘 모델이다.

B2B : 로컬 상점들과 제휴를 맺고 광고성 메시지를 매설할 수 있도록 한다. 이는 CPC방식에서 착안하여 게시물 조회수당으로 광고비를 책정한다. 단순 광고로 끝나지 않고 해당 메시지를 해금하는 유저에게 가게 할인혜택을 제공하여 광고 효과를 극대화 한다.

B2C : 일반 유저의 무분별한 기록을 방지하고 콘텐츠 희소성을 높이기 위해 엽서의 매설 횟수를 1일 1회로 제한한다. 더 많은 기록을 남기고 싶은 헤비유저를 대상으로 DODO 프리미엄 패스를 월정액으로 판매하여 유저에게 더 많은 기록의 기회를 주게 되며 이로 인한 수익화도 가능해 진다.

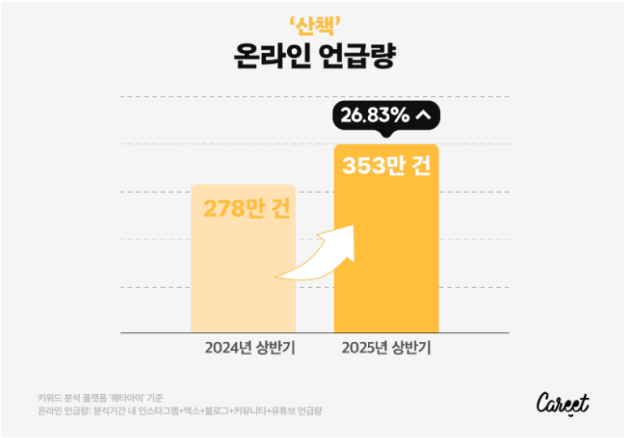
B2G : 지자체 관광 공사와 협력하여 지자체 관광코스를 연동하여 관광객 유치 및 지역 홍보 툴로 사용한다.

2.2.2 시장규모 분석

전체시장(TAM) : 월간 사용률 기준 네이버 지도(3000만), 구글 맵(1000만)으로 네비게이션을 제외한 지도앱 사용자는 4000만명 이상으로 책정된다.

유효시장(SAM) : 가장 자주 쓰이는 SNS 앱인 인스타(2700만) 페이스북(900만)등 SNS의 경우도 한국내에서 활발한 시장임을 알 수 있다.

수익시장(SOM) : 전체 시장과 유효시장의 교집합이 1%만 된다고 가정하여도 약 30만명이다. 1%의 유저층만 초기 타겟으로 잡아도 '산책'이라는 단어의 온라인 내 언급량이 증가(퀘타아이)하고 있는 것을 보아 충분한 시장 자생력, 성장력을 기대할 수 있다.



2.2.3 상용화 시 예상 사업규모

항목	상세 산출 근거	금액 (연간)
B2B 매출	제휴 상점 200곳 * 월 4만원 광고비 (CPC 1회당 10원 * 4000회)	약 0.96억원
B2C 매출	유료 유저 3,000명 (SOM의 1%) * 월 4,900원	약 1.76억원
운영 비용	AWS 서버비(1000만) + 컨텐츠 제작비(1억) + 초기 광고료(1000만)	약 1.2억원
예상 순수익	총 매출(약2.7억) - 비용	약 1.5억원 (영업 이익률 55%)

ARPU(1인당 예상 수익률) : B2B수익 + B2C 수익 / 전체 활성화 유저 수 =
270,000,000원/300,000 = 900원

광고도 주된 수익 중 하나 이므로 무료 이용자들도 소중한 전략적 자산이 된다.

3. User Story Mapping

3.1 페르소나 정의

3.1.1 Persona 1 - 산책자

- **설명**: 일상의 지루함에서 벗어나 동네의 숨겨진 이야기나 실용적인 정보를 찾는 사용자이다. 광고가 아닌 실제 방문자의 진솔한 후기와 재미를 추구한다.
- **목표**
    - 현재 위치 주변의 흥미로운 메시지 및 실용적 정보(지름길, 화장실 등)를 발견하고 싶다.
    - 지루하고 뻔할 수 있는 산책에서 탐험적 재미를 충족하고 싶다.
    - 카테고리별(맛집, 추억, 부모님과 가기 좋은 곳 등) 최신 트렌드로 산책 경로를 탐색하고 싶다.

3.1.2 Persona 2 - 기록자

- **설명**: 자신이 아는 장소의 가치, 특별한 추억, 또는 팁을 공유하며 타인과 정서적으로 교감하고 반응을 얻고 싶은 사용자이다.
- **목표**
    - 직접 가본 장소에 대한 추억을 기반으로 신뢰도 높은 스토리를 게시하고 싶다.
    - 프리뷰 및 블러 설정을 통해 콘텐츠의 희소성과 호기심 유발하고 싶다.
    - 자신이 발행한 엽서가 타인에게 전달되는 과정 및 엽서에 대한 반응을 확인하고 싶다.
    - 광고성 정보와 차별화된 자신만의 로컬 아카이브를 구축하고 싶다.

    - 익명성을 담보로 진솔한 감정을 공유하고 싶다.

3.1.3 Persona 3 - 서비스 관리자

- **설명**: 플랫폼의 신뢰성과 콘텐츠 품질을 유지하며, 지역별 데이터 분석을 통해 서비스 선순환을 관리하는 운영 담당자이다.
- **목표**
    - 부적절한 콘텐츠를 필터링하고 유저 제재를 통해 생태계를 유지하고 싶다.
    - 공지사항을 통해 유저들에게 쉽게 정보를 알리고 싶다.
    - 유저의 문의사항에 답변하며 궁금증을 해소해주고 싶다.
    - 좋아요 지수 및 최신 등록 데이터를 모아보고 싶다.
    - 사용자 활동 데이터 대시보드를 통한 지역 기반 챌린지 및 이벤트 기획하고 싶다.

3.2 사용자 여정 (User Journey)

3.2.1 산책자 Journey

1. **앱 실행 및 설정**: 앱 접속 후 자유 산책 모드 혹은 특정 카테고리(맛집, 풍경 등)를 선택해 필터링한다.
2. **주변 탐색 및 알림**: 지도를 보며 걷다가 알림을 통해 주변의 핀을 발견한다.
3. **프리뷰 확인**: 지도의 핀을 터치해 궁금증을 유발하는 프리뷰를 확인한다.
4. **장소 이동**: 특정 장소를 목적지로 설정하고 GPS 표시를 따라 이동한다.
5. **메시지 해금**: 목적지 반경 내 도착 시 알림과 함께 블러 해제 및 메시지의 전체 스토리를 감상한다.
6. **인터랙션**: 좋아요 버튼 클릭, 댓글 작성 혹은 엽서 교환/수집 참여한다.
7. **여정 완료**: 오늘 다녀온 산책 경로와 수집한 엽서 리스트를 확인하고 저장한다.

3.2.2 기록자 Journey

1. **방문 인증**: 기록을 원하는 장소에서 현재 위치 기반 메시지 작성 버튼 클릭한다.
2. **콘텐츠 작성**: 사진/영상과 함께 해당 장소에 대한 본인의 스토리나 팁을 작성한다.
3. **호기심 설정**: 다른 유저의 방문을 유도하기 위해 메시지의 블러 해금 범위를 설정한다.

4. **카테고리 및 엽서 등록**: 적절한 카테고리 태그를 선택하고, 필요 시 엽서 아이템도 등록한다.
5. **등록 완료**: 중복 등록 여부 및 가이드라인 준수 확인 후 메시지를 업로드한다.
6. **반응 모니터링**: 내가 메시지를 작성한 장소를 방문한 인원수, 좋아요 수, 댓글 및 엽서 전달 여부를 확인한다.

3.2.3 관리자 Journey

1. **콘텐츠 모니터링**: 대시보드를 통해 신고된 게시물 및 부적절한 콘텐츠를 필터링하고 검수한다.
2. **노출 제어**: 좋아요 점수가 낮거나 오래된 메시지의 노출 우선순위를 조정하여 콘텐츠 신선도를 유지한다.
3. **유저 관리**: 플랫폼 가이드 위반 유저에 대한 일시 정지 또는 차단 처리를 한다.
4. **데이터 분석**: 지역별/카테고리별 활동 데이터를 분석하여 인기 테마 산책 경로 업데이트하고 광고를 관리한다.

3.3 Epics & User Stories

3.3.1 Epic 0: 앱 실행 및 회원가입&로그인

- **User Story 0.1**: 나는 사용자로서 앱 실행 시 소셜 로그인으로 회원가입 하고싶다.
- **User Story 0.2**: 나는 사용자로서 회원가입한 소셜 아이디로 로그인을 하고싶다.

3.3.2 Epic 1: 경로 탐색 및 트래킹

- **User Story 1.1**: 나는 산책자로서, 앱 실행 시 현재 위치 주변의 메시지를 확인하고 싶다. → 어떤 흥미로운 지점이 있는지 파악하기 위해
- **User Story 1.2**: 나는 산책자로서, 특정 카테고리나 테마(예: #부모님, #고양이)를 필터링하고 싶다. → 내 현재 목적에 맞는 장소(메시지)를 찾기 위해
- **User Story 1.3**: 나는 산책자로서, 이동 중 근처의 인기 메시지를 실시간 알림으로 받고 싶다. → 앱을 계속 보지 않아도 새로운 발견을 하기 위해 (백그라운드 알림)

3.3.3 Epic 2: 로컬 메시지 생성 및 관리

- **User Story 2.1**: 나는 기록자로서, 특정 장소에 관한 사진과 글을 등록하고 싶다. → 장소에 기반한 스토리 또는 팁을 다른 사람들과 공유기 위해

- **User Story 2.2**: 나는 기록자로서, 해당 위치에 메시지 해금될 거리 범위를 설정하고 싶다.→ 내 콘텐츠에 진심으로 관심 있는 사람만 내용을 확인하게 하기 위해
- **User Story 2.3**: 나는 광고주로서, 광고 권한을 얻고 광고 게시물을 작성 하고 싶다.

3.3.4 Epic 3: 위치 기반 콘텐츠 해금

- **User Story 3.1**: 나는 산책자로서, 메시지 선택한 지점까지의 정확한 위치를 알고 싶다. → 숨겨진 정확한 위치를 찾아가기 위해 + 숨겨진 내용을 확인하기 위해

3.3.5 Epic 4: 인터랙션 및 마이페이지&엽서 시스템

- **User Story 4.1**: 나는 산책자로서, 메시지 확인 후 Hype(추천)/비추천, 댓글을 남기고 싶다. → 기록자에게 피드백을 주고 내 활동을 기록하기 위해
- **User Story 4.2**: 나는 산책자 및 기록자로서, 특정 장소에서만 얻을 수 있는 일회성 엽서를 수집하거나 교환하고 싶다. → 산책의 경험을 소장 가능한 보상으로 남기기 위해
- **User Story 4.3**: 나는 기록자로서 마이페이지에서 나의 게시물에 대한 활동과 다른 유저들의 반응을 확인하고 싶다. → 나의 경험을 한 눈의 확인 및 관리하고 다른 유저들의 반응을 보기위해
- **User Story 4.4**: 나는 기록자로서, 엽서 전달 여부 및 내 엽서를 받은 유저의 간단한 반응을 확인하고 싶다. → 타인과의 느슨한 연결감을 느끼기 위해

3.3.6 Epic 5: 운영 및 품질 관리

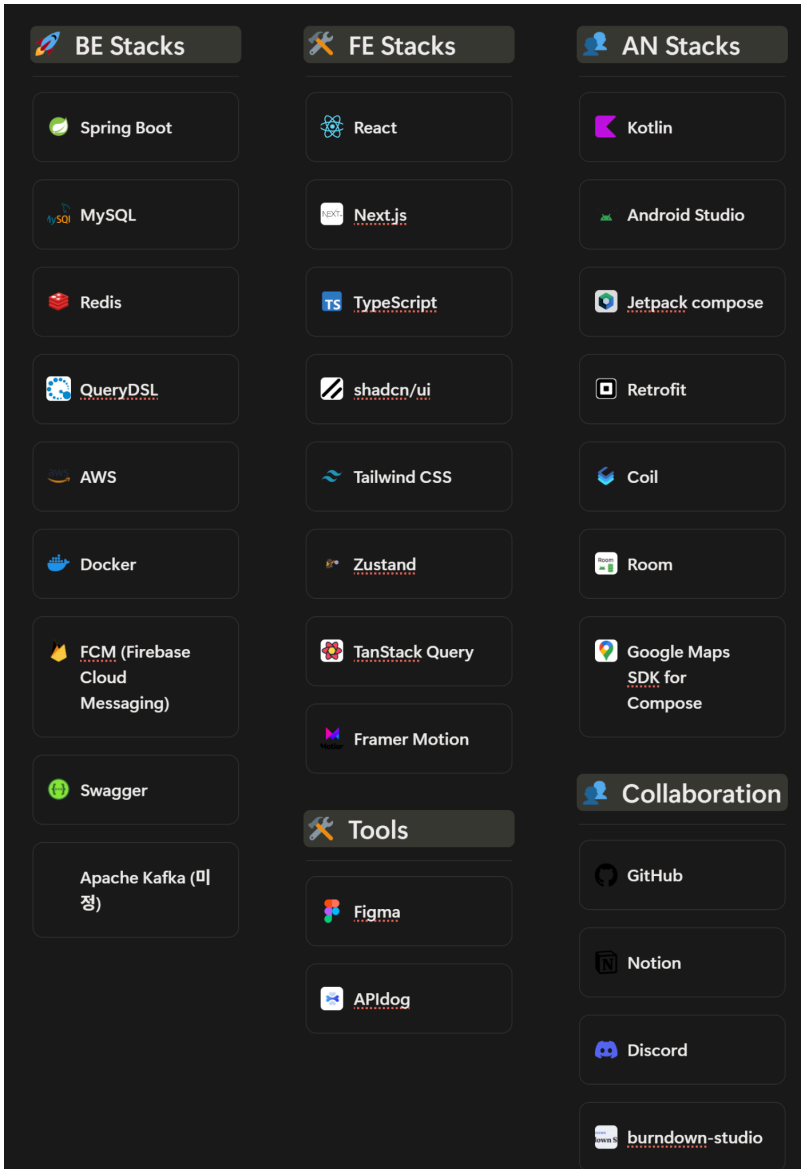
- **User Story 5.1**: 나는 관리자로서, 부적절한 게시물, 댓글, 엽서를 필터링하고 신고된 유저를 제재하고 싶다. → 청결한 환경을 만들기 위해
- **User Story 5.2**: 나는 관리자로서, 카테고리를 관리 할 수 있어야 한다. → 유저 트렌드에 맞는 카테고리를 유지하기 위해
- **User Story 5.3**: 나는 관리자로서, 중요한 공지사항을 전체 사용자에게 알림 보낼 수 있어야 한다. → 유저가 정보 부족으로 힘들어 하지 않게 하기 위해
- **User Story 5.4**: 나는 관리자로서, 게시물, 댓글, 엽서 관련 통계 자료를 확인하고 싶다. → 더 나은 품질의 앱을 위해
- **User Story 5.5**: 나는 관리자로서, 유저의 문의사항을 확인하고 처리하고 싶다. → 유저들의 니즈를 파악하기 위해
- **User Story 5.6**: 나는 관리자로서, 광고 게시물 관리, 광고주 관리를 할 수 있어야한다. →

더나은 광고 환경을 제공하기 위해

4. 설계 및 구현

4.1 기술 분석

4.1.1 기술 스택



4.1.1.1 Backend Stack

1. Spring Boot

- **(1) 활용 목적**: 프로젝트의 백엔드 메인 프레임워크로 활용하며, 비즈니스 로직 처리 및 RESTful API 서버 구축을 담당한다.
- **(2) 선정 이유**: 대규모 라이브러리 생태계와 안정적인 Spring Security 보안 환경을 활용하여 대용량 트래픽 환경에서도 유지보수 편의성을 극대화할 수 있기 때문이다.

2. MySQL

- **(1) 활용 목적**: 관계형 데이터베이스(RDB)로서 사용자 정보, 메시지 데이터, 엽서 기록 등 정형화된 데이터의 영구 저장소로 활용한다.
- **(2) 선정 이유**: NoSQL 대비 데이터 무결성 보장이 뛰어나며, 특히 위치 기반 서비스에 필수적인 ST_Distance_Sphere 와 같은 공간 함수를 기본 지원하여 위경도 기반의 반경 검색 및 위치 데이터 쿼리 최적화에 가장 적합하기 때문이다.

3. Redis

- **(1) 활용 목적**: 인메모리 데이터 저장소로서 메시지 조회수 실시간 집계 및 유저의 메시지 해금 여부 확인 등 빈번한 I/O가 발생하는 데이터의 캐싱 레이어로 활용한다.
- **(2) 선정 이유**: 디스크가 아닌 메모리 기반 저장소로 응답 속도가 마이크로초(μs) 단위로 매우 빠르며, MySQL의 부하를 획기적으로 낮추어 전체 시스템의 성능 병목 현상을 방지할 수 있기 때문이다.

4. QueryDSL

- **(1) 활용 목적**: 복잡한 동적 쿼리 및 다중 필터링 로직을 자바 코드로 구현하기 위해 활용한다.
- **(2) 선정 이유**: 문자열 기반의 JPQL과 달리 자바 코드로 작성되어 컴파일 시점에 에러를 발견할 수 있다. 이는 오타로 인한 런타임 에러를 방지하고, 조건이 복잡한 검색 쿼리에서도 코드 가독성과 유지보수성을 극대화하기 때문이다.

5. AWS (EC2, S3, CloudFront)

- **(1) 활용 목적**: 클라우드 컴퓨팅 인프라 및 대용량 정적 자원(이미지) 저장 및 전송 환경으로 활용한다.

- **(2) 선정 이유**: 고도의 가용성과 글로벌 확장성을 제공하며, 특히 CloudFront(CDN)를 결합하여 엽서 이미지 전송 시 S3 원본 서버의 부하를 줄이고 에지 로케이션을 통한 전송 속도 최적화 및 비용 절감이 가능하기 때문이다.

6. Docker

- **(1) 활용 목적**: 애플리케이션의 실행 환경을 컨테이너화하여 개발, 테스트, 운영 환경의 일관성을 유지하기 위해 활용한다.
- **(2) 선정 이유**: 가상 머신(VM) 대비 오버헤드가 적고 배포 속도가 빠르며, 개발 환경 불일치 문제를 근본적으로 해결하여 인프라 관리 및 배포 효율을 높여주기 때문이다.

7. FCM (Firebase Cloud Messaging)

- **(1) 활용 목적**: 안드로이드 단말기에 실시간 엽서 도착 알림 및 서비스 공지 사항을 푸시 알림으로 전송하기 위해 활용한다.
- **(2) 선정 이유**: 별도의 소켓 서버를 유지할 필요 없이 구글의 검증된 인프라를 활용하여 배터리 소모를 최소화하면서도 저비용으로 높은 도달률의 실시간 메시지 전송이 가능하기 때문이다.

8. Swagger

- **(1) 활용 목적**: 백엔드 API 명세서를 자동으로 생성하고 관리하기 위해 활용한다.
- **(2) 선정 이유**: Spring Rest Docs 대비 설정이 간편하며 테스트 코드를 완벽하게 작성하지 않아도 실시간으로 API 명세가 갱신되기 때문이다.

9. Apache Kafka (도입 검토 중)

- **(1) 활용 목적**: 대규모 트래픽 발생 시 시스템의 완충 역할을 수행하여 안정성을 높이기 위해 활용한다.
- **(2) 선정 이유**: 추후 부하 테스트를 통해 트래픽 급증 시 서버 다운을 방지하는 완충 역할을 수행한다.

4.1.1.2 Frontend Stack

1. React

- **(1) 활용 목적**: 사용자 인터페이스(UI) 구축을 위한 핵심 라이브러리로 활용하며, 컴포넌트 기반 아키텍처를 통해 관리자 페이지와 앱뷰 간의 UI 재사용성을 극대화한다.

- **(2) 선정 이유**: 방대한 커뮤니티 생태계와 검증된 안정성을 바탕으로 선언적 프로그래밍 모델을 제공하여, 복잡한 상태 변화에 따른 UI 업데이트를 효율적으로 처리하고 개발 생산성을 높일 수 있기 때문이다.

2. Next.js (App Router)

- **(1) 활용 목적**: 프론트엔드 프레임워크로서 서버 사이드 렌더링(SSR) 및 정적 사이트 생성(SSG)을 지원하며, 관리자 전용 경로와 앱뷰 경로를 효율적으로 라우팅한다.
- **(2) 선정 이유**: SEO는 물론, 이미지 최적화(Next/Image)와 코드 분할(Code Splitting) 기능을 통해 모바일 네트워크 환경의 앱뷰 사용자에게 빠른 초기 로딩 속도를 제공하기 때문이다. 또한 관리자 페이지는 유저 정보 같은 민감한 데이터를 사용하므로 Server Components를 사용하면 브라우저에 소스코드를 다 노출하지 않고 화면만 보내줄 수 있어 구조적으로 안전하기 때문이다.

3. TypeScript

- **(1) 활용 목적**: 자바스크립트에 정적 타입을 도입하여 개발 단계에서 데이터 타입을 명시하고 코드의 안정성을 확보한다.
- **(2) 선정 이유**: 백엔드(Spring)와의 API 통신 시 DTO 구조를 타입으로 정의하여 런타임 에러를 사전에 방지하고, IDE의 자동 완성 기능을 통해 협업 과정에서의 코드 가독성과 유지보수성을 극대화하기 때문이다.

4. shadcn/ui

- **(1) 활용 목적**: Radix UI와 Tailwind CSS를 기반으로 한 고품질의 UI 컴포넌트를 프로젝트 내부에 직접 구성하여 일관된 디자인 시스템을 구축한다.
- **(2) 선정 이유**: 라이브러리 설치 방식이 아닌 소스 코드 복사 방식을 채택하여 커스텀 자유도가 매우 높으며, 관리자 페이지의 복잡한 Table이나 웹뷰의 Drawer(바텀 시트) 등을 가장 빠른 속도로 구현할 수 있기 때문이다.

5. Tailwind CSS

- **(1) 활용 목적**: 유틸리티 우선(Utility-first) CSS 프레임워크로, 별도의 CSS 파일 없이 HTML 클래스 내에서 스타일링을 즉시 적용한다.
- **(2) 선정 이유**: 클래스명 명명 규칙에 대한 고민을 줄여 개발 속도를 높이고, 빌드 시 사용하지 않는 스타일을 제거하여 최종 번들 크기를 최소화함으로써 저사양 모바일 기기에서의 렌더링

성능을 최적화하기 때문이다.

6. Zustand

- **(1) 활용 목적**: 클라이언트 사이드의 전역 상태 관리(로그인 상태, 유저 설정 등)를 위한 경량 상태 관리 라이브러리로 활용한다.
- **(2) 선정 이유**: Redux 대비 보일러플레이트 코드가 거의 없어 학습 곡선이 낮고, React의 훅(Hook) 시스템과 완벽하게 호환되어 관리자와 앱뷰 전체에서 필요한 전역 상태를 가장 직관적으로 제어할 수 있기 때문이다.

7. TanStack Query (React Query)

- **(1) 활용 목적**: 서버 데이터 페칭, 캐싱, 동기화 및 업데이트를 관리하는 서버 상태 관리 라이브러리로 활용한다.
- **(2) 선정 이유**: API 호출 결과의 자동 캐싱 및 백그라운드 업데이트 기능을 통해 불필요한 네트워크 요청을 줄이고, 로딩 및 에러 상태를 선언적으로 관리하여 유저에게 끊김 없는 데이터 조회 경험을 제공하기 때문이다.

8. Framer Motion(도입 검토 중)

- **(1) 활용 목적**: 웹뷰 페이지 전환 및 인터랙티브한 UI 요소에 애니메이션 효과를 부여한다.
- **(2) 선정 이유**: 단순한 CSS 애니메이션보다 세밀한 제어가 가능하며, 특히 모바일 웹뷰 환경에서 사용자가 실제 네이티브 앱을 사용하는 것과 같은 부드러운 사용자 경험(UX)을 제공하는 데 탁월하기 때문이다.

4.1.1.3 Android Native

1. Kotlin

- **(1) 활용 목적:** 안드로이드 애플리케이션의 비즈니스 로직 구현 및 데이터 처리를 위한 메인 프로그래밍 언어로 활용한다.
- **(2) 선정 이유:** 구글이 권장하는 'Kotlin First' 표준에 따라 Null Safety 기능을 통해 앱의 안정성을 확보하고, Java 대비 간결한 문법으로 코드 양을 줄여 개발 생산성을 극대화할 수 있고 최신 트렌드가 Kotlin으로 하는 성향이 있다.

2. MVVM

- **(1) 활용 목적:** UI 로직과 비즈니스 로직을 분리하는 표준 아키텍처로 활용하여, 각 계층의 독립성을 보장하고 코드의 가독성을 높인다.
- **(2) 선정 이유:** 데이터의 변화를 관찰하여 UI를 자동으로 업데이트하는 반응형 구조에 최적화되어 있으며, 기능 확장 시 다른 계층에 영향을 주지 않고 유지보수가 용이하다.

3. Jetpack Compose

- **(1) 활용 목적:** 선언형 UI 프레임워크로서 '메시지', '엽서' 등 서비스 특유의 감성적인 사용자 인터페이스를 구축하는 데 활용한다.
- **(2) 선정 이유:** 기존 XML 방식보다 코드 직관성이 높고 상태(State) 관리가 용이하며, 복잡한 애니메이션 효과를 적은 코드로 정교하게 구현하여 사용자 경험(UX)을 향상시킬 수 있기 때문이다.

4. RETROFIT & OKHTTP

- **(1) 활용 목적:** 서버와의 REST API 통신 및 이미지(Multipart) 업로드 처리를 위한 네트워크 클라이언트로 활용한다.
- **(2) 선정 이유:** 인터페이스 기반의 직관적인 설정으로 네트워크 로직을 규격화할 수 있으며, OkHttp 인터셉터를 통해 토큰 인증 및 로그 관리를 중앙 집중식으로 처리하여 보안성과 효율성을 동시에 확보할 수 있기 때문이다.

5. COROUTINE & FLOW

- **(1) 활용 목적:** 네트워크 통신 및 DB 작업 등의 무거운 작업을 백그라운드에서 처리하고, 데이터의 실시간 흐름을 UI에 전달하는 비동기 프레임워크로 활용한다.
- **(2) 선정 이유:** 비동기 코드를 동기 코드처럼 읽기 쉽게 작성할 수 있어 협업 시 가독성이 뛰어나고, 리소스 소모가 적어 모바일 환경에서 고성능 데이터 스트리밍을 구현하기 적합하기 때문이다.

6. ROOM

- **(1) 활용 목적:** 오프라인 환경에서도 서비스 이용이 가능하도록 '수집한 엽서'나 '메시지 목록' 등의 데이터를 기기 내부에 저장하는 로컬 데이터베이스로 활용한다.
- **(2) 선정 이유:** SQLite의 기능을 추상화하여 SQL 쿼리를 컴파일 타임에 검증함으로써 런타임 에러를 방지하고, 코루틴과의 긴밀한 연동을 통해 안정적인 데이터 로딩을 지원하기 때문이다.

7. COIL

- **(1) 활용 목적:** 서버 URL에 저장된 이미지 데이터를 비동기적으로 로딩하여 지도 핀이나 엽서 목록에 렌더링하는 데 활용한다.
- **(2) 선정 이유:** 코틀린 코루틴 기반으로 설계되어 메모리 효율이 뛰어나며, Jetpack Compose와의 높은 호환성을 바탕으로 불필요한 리소스 낭비 없이 빠른 이미지 렌더링을 제공하기 때문이다.

8. Google Maps SDK for Compose

- **(1) 활용 목적:** 사용자 위치 기반의 '메시지' 탐험 및 매설 기능을 구현하기 위한 핵심 지도 라이브러리로 활용한다.
- **(2) 선정 이유:** 가장 방대한 전 세계 지도 데이터를 보유하고 있으며, 컴포즈 전용 라이브러리를 통해 지도 인터랙션과 UI 요소를 선언형 방식으로 일관성 있게 제어할 수 있기 때문이다.

4.1.2 개발 및 협업 환경

개발 환경

- IntelliJ IDEA: 자바 및 스프링 부트 개발에 최적화된 통합 개발 환경(IDE)
- Android Studio : 안드로이드 native 앱 개발
- Visual Studio Code: 프론트엔드 애플리케이션 개발을 위한 메인 통합 개발 환경(IDE)

협업 환경

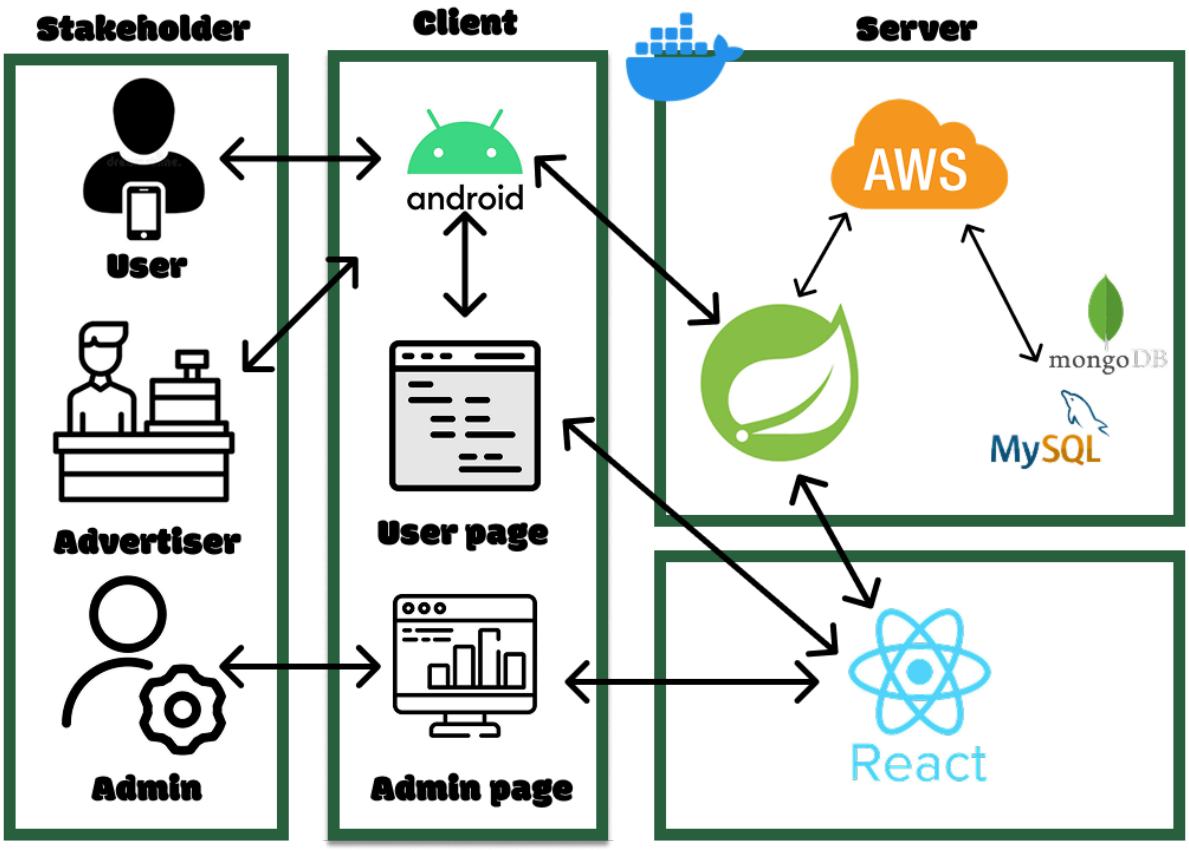
- APIdog (API 설계, 명세화, 디버깅 및 테스트를 통합 관리)
- discord (데일리 스크럼 및 비대면 회의, 깃허브 웹훅)
- GitHub Organization (코드 리뷰, 이슈 트래킹)
- notion (문서 작성, 회의록 관리)
- burndown-studio (애자일 개발 계획 수립)

CI/CD Tools

- github action (CI/CD 파이프라인의 핵심 도구로 활용)
- Vercel (프론트엔드 앱의 메인 호스팅 및 배포 플랫폼으로 활용)

4.2 시스템 설계

4.2.1 시스템 개요

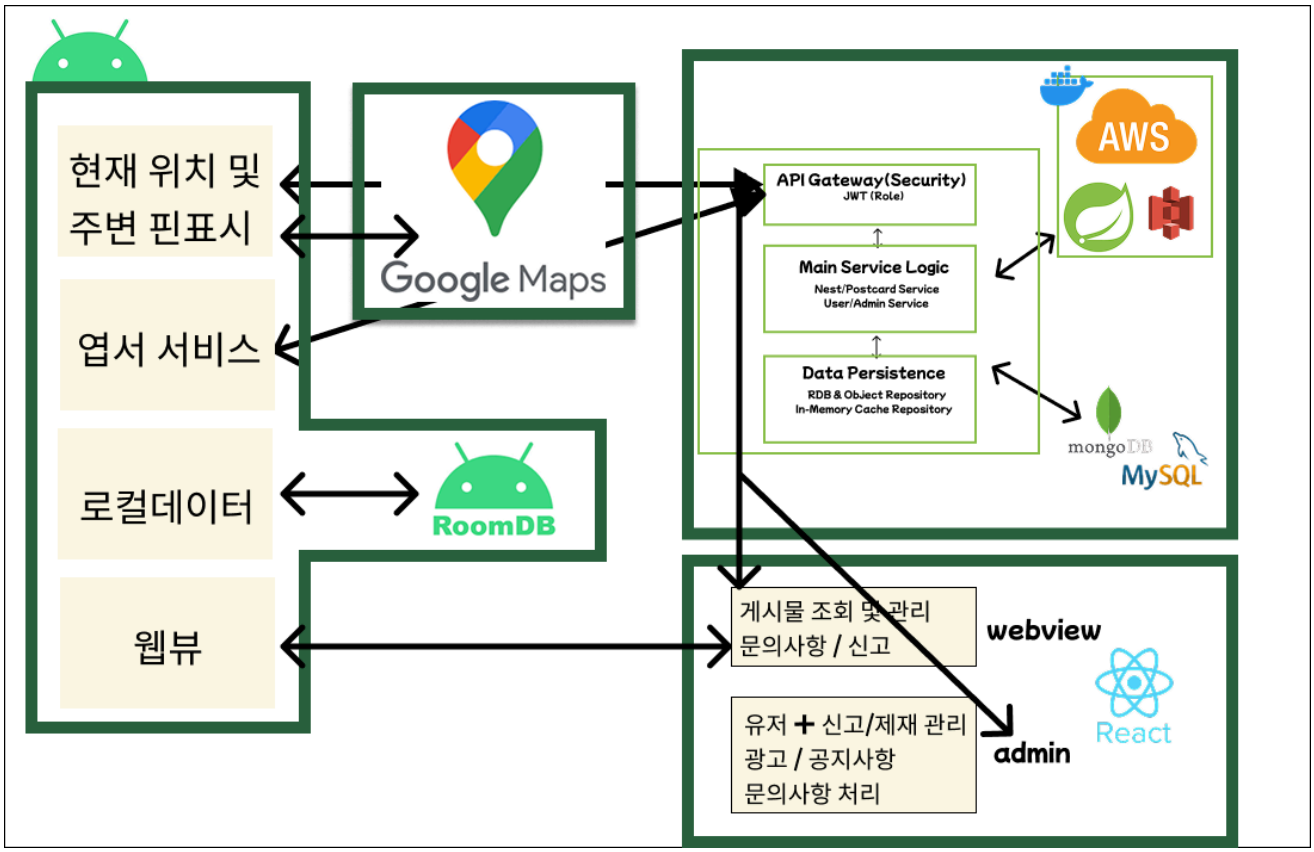


Stakeholder 중 유저와 광고주는 앱을 통해 서비스를 접한다. 일반 유저는 메시지를 탐색하고 작성할 수 있으며, 광고주는 앱을 통해 광고 작성을 할 수 있다. 관리자는 웹 화면으로 서비스를 관리한다.

Client를 구성하는 Android와 React는 Spring을 통해 Backend 서버와 통신하며 데이터에 접근한다.

React는 관리자 웹페이지와 유저 페이지의 일부를 구현한다. 유저 페이지는 웹뷰로 유저와 광고주가 사용하는 앱 화면에 표시된다.

4.2.2 소프트웨어 아키텍처



Android

- 현재 위치 및 주변 핀표시는 Google Maps Api의 여러 기능을 활용하여 구현한다.
- 현재 위치, 주변 핀 및 엽서 서비스는 자체 UI로 구현하며 API gateway를 통해 서버의 db와 통신한다.
- 로컬데이터 중 무거운 데이터는 RoomDB를 간단한 데이터는 sharedpreference에 저장한다.
- 마이페이지, 게시물, 카테고리 등의 화면은 React로 만들어진 userpage를 웹뷰로 띄워준다.

Frontend

클라이언트 중 React에서는 API를 통해 서버와 통신하며 웹 브라우저에서 사용될 admin 페이지와

앱에서 webview로 보여질 페이지 2가지를 구현한다.

- Admin - 관리자에게 유저 관리와, 문의사항 처리, 공지사항, 광고, 신고/제재 관리 서비스를 제공한다.
- Webview(User) - 게시물 조회 및 관리, 문의사항 작성, 신고 기능을 구현해 안드로이드 화면에서 보여준다.

Backend

백엔드는 Spring Boot 프레임워크를 기반으로 하며, 서비스의 보안, 비즈니스 로직 처리, 데이터 영속성 관리를 위해 세 가지 핵심 계층으로 분리하여 설계한다.

API Gateway (Security Layer)

- 외부 클라이언트(Android, React Admin)로부터 들어오는 모든 요청의 관문 역할을 수행한다.
- 모든 http 요청 헤더의 JWT(JSON WEB TOKEN)를 검증하고 유저 권한에 따른 접근 제어를 수행한다.

Main Service Logic (Business Layer)

- 애플리케이션의 핵심 비즈니스 요구사항을 처리하는 계층이다.
- **Nest/Postcard Service:** 좌표 기반의 메시지 해금 로직 검증, 엽서 송수신등 비즈니스 흐름을 제어한다.
    - 메세지 해금의 경우 사용자의 현재위치와 데이터 베이스의 게시물 위치 좌표 거리를 계산하여 해금 가능한지 여부를 검증한다.
    - 엽서의 경우 엽서가 등록되어 있는지 여부를 검증하여 같은 엽서가 중복 등록 되지 않도록 검증한다.
- **User/Admin Service:** 회원 상태 관리 및 관리자 페이지(React)에서 호출하는 신고/제재 처리, 광고 관리 등의 행정 로직을 수행한다.

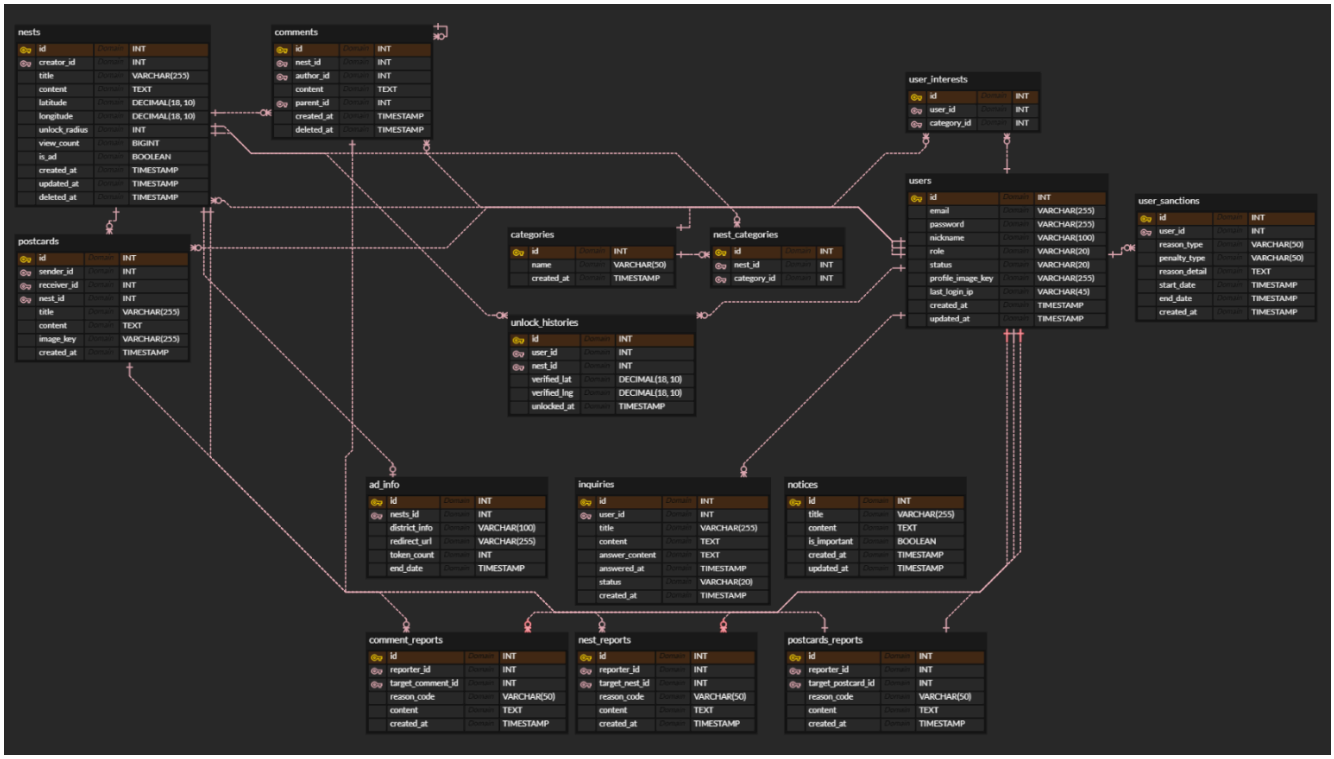
Data Persistence (Infrastructure Layer)

- 비즈니스 로직에서 처리된 데이터를 각 저장소의 특성에 맞게 저장하고 조회하는 계층이다.
- **RDB & Object Repository:** MySQL을 사용하여 사용자 및 게시물의 영속성을 관리하며, S3와 연동하여 미디어 데이터의 경로(Object Key)를 매핑한다.
- **In-Memory Cache Repository:** Redis를 활용하여 실시간 조회수 카운팅(INCR), 중복 조회

방지, JWT Refresh 토큰 캐싱등을 처리하여 메인 DB의 부하를 줄인다.

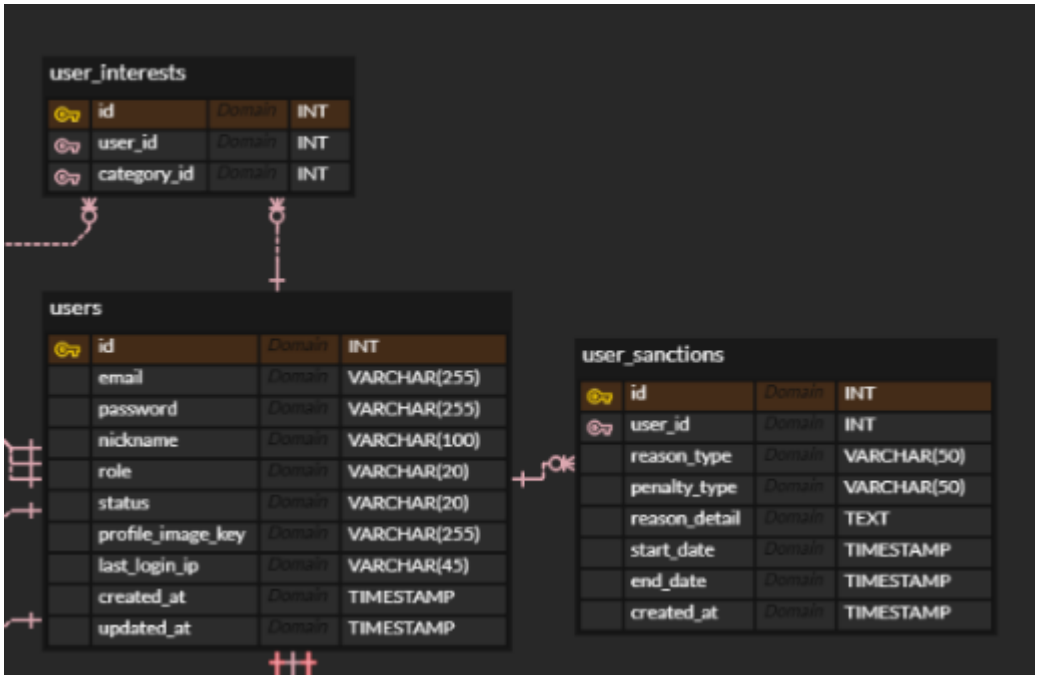
4.2.3 데이터 설계

본 절에 제시된 ERD 및 데이터베이스 스키마는 초기 도메인 설계를 바탕으로 작성되었으며, 실제 개발 단계에서의 데이터 정규화, 인덱싱 최적화 및 쿼리 성능 개선을 위해 테이블 구조나 관계 설정이 일부 변경될 수 있다.



ERD 전체 구조도

4.2.3.1 사용자 관리



사용자 데이터는 서비스 권한 인증 및 모든 활동의 핵심이 된다.

users

- 필드에는 id(PK), email, password, nickname, role, status, profile_image_key, last_login_ip, created_at, updated_at이 있다.
- 시스템내의 모든 활동(Nest 생성, 댓글, 신고 등)의 주체가 된다.

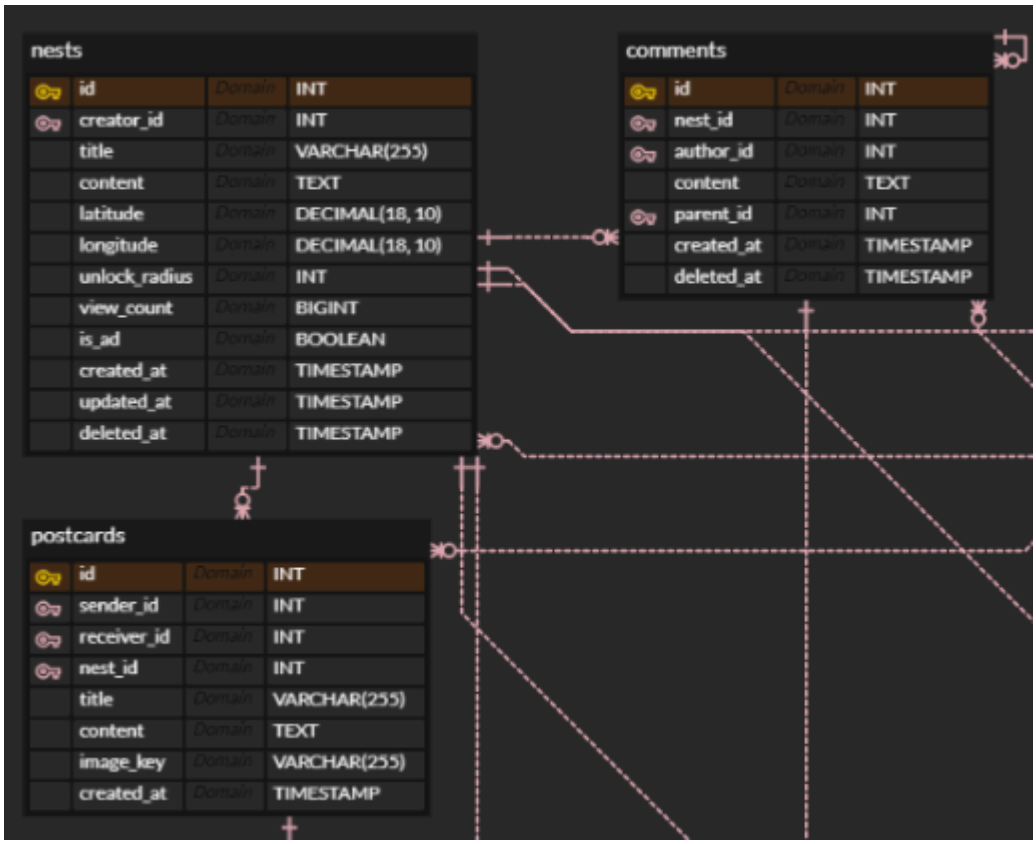
user_interests

- 필드에는 id(PK), user_id(FK), category_id(FK)가 있다.
- users : user_interests를 1:N으로 하여 한명의 사용자가 여러개의 카테고리를 갖게 된다.
- categoriest : user_interests를 1:N으로 하여 하나의 카테고리가 여러 사용자에 의해 선택될 수 있도록 한다.
- users : categories를 M : N으로 할 수 있게 해준다.

user_sanctions

- 필드에는 id(PK), user_id(FK), reason_type, penalty_type, reason_detail, start_date, end_date, created_at이 있다.
- users : user_sanctions를 1:N으로 하여 한명의 사용자가 여러 제재를 받을 수 있게 된다.

4.2.3.2 핵심 콘텐츠



위치 기반 데이터와 사용자 간의 콘텐츠 전송을 정의한다.

nests

- 필드에는 id(PK), creator_id(FK), title, content, latitude, longitude, unlock_radius, view_count, is_ad, created_at, updated_at, deleted_at이 있다.
- users : nests를 1 : N으로 하여 한명의 사용자가 여러 글을 작성할 수 있도록 한다.

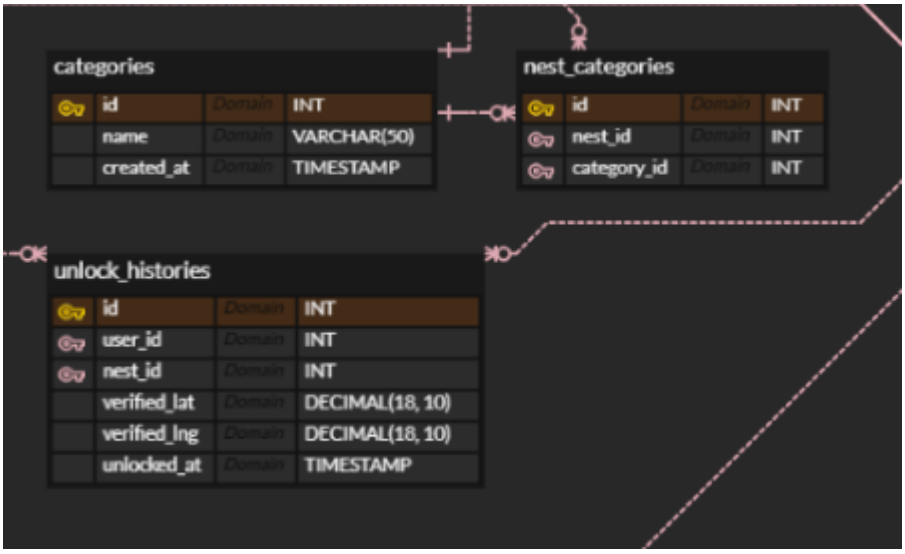
postcards

- 필드에는 id(PK), sender_id(FK), receiver_id(FK), nest_id(FK), title, content, image_key, created_at이 있다.
- users : postcards를 1 : N으로 하여 한명의 사용자가 여러 엽서를 작성할 수 있도록 한다.(sender)
  users : postcards를 1 : 1으로 하여 하나의 엽서는 한명만 받을 수 있도록 한다.(receiver)
  nests : postcards를 1 : N으로 하여 하나의 메시지에 여러 엽서가 거쳐 갈 수 있도록 한다.

comments

- 필드에는 id(PK), nest_id(FK), author_id(FK), content, parent_id(FK), created_at, deleted_at이 있다.
- nests : comments를 1 : N으로 하여 하나의 게시물에 여러 댓글이 적힐 수 있도록 한다.
- users : comments를 1 : N으로 하여 한명의 사용자가 여러 댓글을 작성할 수 있도록 한다.
- comments : comments를 1: N으로 하여(self-joined) 하나의 댓글에 여러 대댓글이 달릴 수 있도록 한다.

4.2.3.3 위치 및 카테고리



실제 위치기반 활동과 카테고리 분류를 정의한다.

unlock_histories

- 필드에는 id(PK), user_id(FK), nest_id(FK), verified_lat, verified_lng, unlocked_at이 있다.
- nests : unlock_histories를 1:N으로 하여 하나의 게시물에 여러 해금기록이 생길 수 있도록 한다.
- users : unlock_histories를 1:N으로 하여 한명의 사용자가 여러 해금기록을 가질 수 있도록 한다.
- users와 nests가 M:N으로 할 수 있게 해준다.

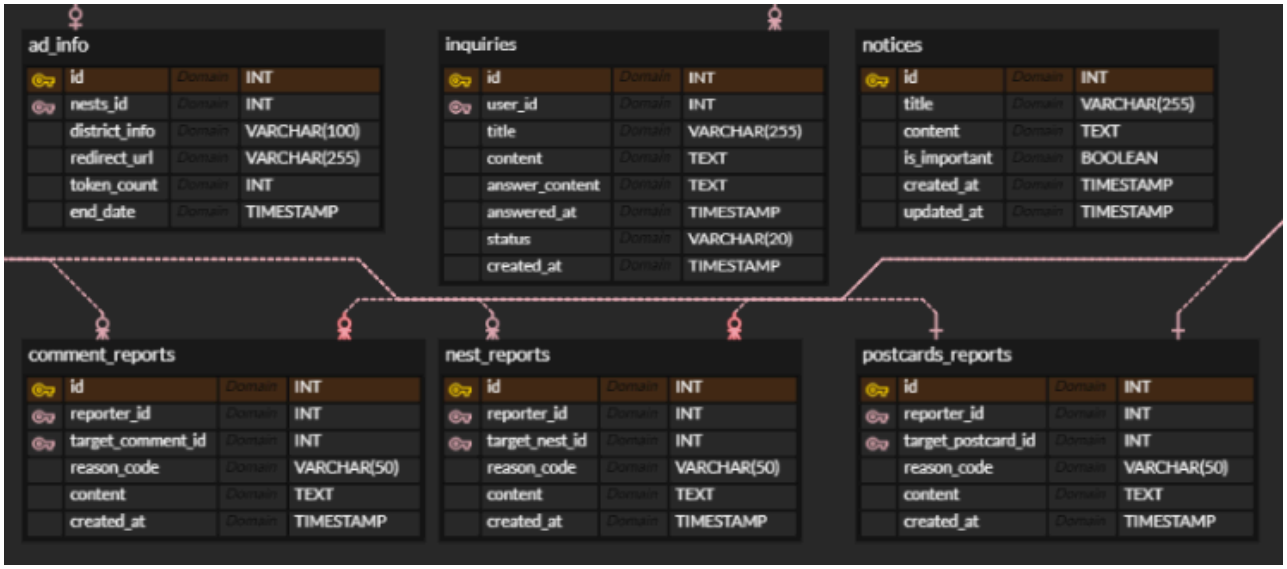
categories

- 필드에는 id(PK), name, created_at이 있다.
- 시스템내의 모든 카테고리(Nest, User)의 주체가 된다.

nest_categories

- 필드에는 id(PK), nest_id(FK), category_id(FK)가 있다.
- nests : nest_categories를 1:N으로 하여 하나의 게시물에 여러 카테고리를 가질 수 있도록 한다.
- categories : nest_categories를 1:N으로 하여 하나의 카테고리가 여러 게시물에 의해 선택될 수 있도록 한다.
- nests와 categories를 M:N으로 할 수 있게 해준다.

4.2.3.4 운영



서비스의 지속 가능성을 위한 광고, 문의, 신고를 관리한다.

ad_info

- 필드에는 id(PK), nest_id(FK), district_info, redirect_url, token_count, end_date가 있다.
- nests : ad_info를 1:1으로 하여 하나의 핀에 하나의 광고정보가 올라 올 수 있도록 한다.

inquiries

- 필드에는 id(PK), user_id(FK), title, content, answer_content, answered_at, status, created_at이 있다.
- users : inquiries를 1:N으로 하여 한명의 사용자가 여러 문의사항을 넣을 수 있다.

nest_reports

- 필드에는 id(PK), reporter_id(FK), target_nest_id(FK), reason_code, content, created_at이 있다.
- users : nest_reports를 1:N으로 하여 한명의 사용자가 여러 신고를 넣을 수 있다.

- nests : nest_reports를 1:N으로 하여 하나의 게시물에 여러 신고가 들어 올 수 있다.

comment_reports

- 필드에는 id(PK), reporter_id(FK), target_comment_id(FK), reason_code, content, created_at이 있다.
- users : comment_reports를 1:N으로 하여 한명의 사용자가 여러 신고를 넣을 수 있다.
- comments : comment_reports를 1:N으로 하여 하나의 댓글에 여러 신고가 들어 올 수 있다.

postcard_reports

- 필드에는 id(PK), reporter_id(FK), target_postcard_id(FK), reason_code, content, created_at이 있다.
- users : postcard_reports를 1:1으로 하여 한명의 사용자가 하나의 신고를 넣을 수 있다.
- postcards : postcard_reports를 1:1으로 하여 하나의 엽서에 하나의 신고가 들어 올 수 있다.

notices

- 필드에는 id(PK), title, content, is_important, created_at, updated_at이 있다.
- 독립 엔티티 이다.

4.2.4 API 정의

본 절에 기재된 API 명세는 설계 단계의 초안이며, 실제 구현 및 통합 과정에서 효율적인 데이터 처리와 예외 케이스 대응을 위해 일부 엔드포인트, 요청 파라미터 및 응답 구조가 조정될 수 있다.
메시지(Nest) 관리 API

사용자가 특정 장소에 메시지를 생성하거나, 해당 메시지의 상세 정보를 조회하기 위한 API이다.

- Endpoint: POST /api/v1/nests
- Request Body:

```
{
  "title": "아주대 삼거리 야경 맛집",
  "content": "여기서 보는 일몰이 정말 예뻐요. 꼭 해금해서 확인해보세요!",
  "latitude": 37.2798,
  "longitude": 127.0436,
  "unlockRadius": 50,
  "categoryIds": [1, 3]
}
```

- Response Body:

```
{
  "nickname": "도도대장",
  "favoriteCategories": [1, 3, 5],
  "fcmToken": "fcm-token-string-abcd-1234"
  "profileImageUrl": "https://lh3.googleusercontent.com/..."
}
```

메시지 해금(Unlock) API

사용자의 실시간 위경도 좌표를 서버의 메시지 좌표와 비교하여, 설정된 반경 이내일 경우 메시지 접근 권한을 부여하는 API이다.

- Endpoint: POST /api/v1/nests/{nestId}/unlock
- Request Body:

```
{
  "userLatitude": 37.2799,
  "userLongitude": 127.0437
}
```

- Response Body:

```
{
  "status": "SUCCESS",
  "data": {
    "nestId": 101,
    "unlockedAt": "2026-04-01T17:05:00",
    "distance": 12.5
  },
  "message": "성공적으로 해금되었습니다! 이제 둥지의 엽서를 확인할 수 있습니다."
}
```

카테고리별 메시지 필터링 및 리스트 조회 API

지도 중심점 혹은 사용자 위치를 기준으로 특정 카테고리에 속한 주변 메시지 목록을 검색 반경 내에서 페이징하여 조회하는 API이다.

- Endpoint: GET /api/v1/nests (Query Parameters: categoryId, latitude, longitude, radius 등)
- Query Parameters:
  - categoryId (Long): 필터링할 카테고리 ID (필수)
  - latitude (Decimal): 현재 지도 중심 위도 (선택)
  - longitude (Decimal): 현재 지도 중심 경도 (선택)
  - radius (Integer): 검색 반경 (m 단위, 기본값 500)
  - page (Integer): 페이지 번호 (기본값 0)
  - size (Integer): 한 페이지당 개수 (기본값 20)
- Example: /api/v1/nests?categoryId=1&latitude=37.2801&longitude=127.0440&radius=1000&page=0&size=10
- Response Body:

```
{
  "status": "SUCCESS",
  "data": {
    "content": [
      {
        "nestId": 101,
        "title": "아주대 정문 맛집 둥지",
        "categoryName": "맛집",
        "latitude": 37.2801,
        "longitude": 127.0440,
        "distance": 150.5,
        "isUnlocked": true,
        "viewCount": 1240
      },
      {
        "nestId": 105,
        "title": "숨겨진 디저트 카페",
        "categoryName": "맛집",
        "latitude": 37.2815,
        "longitude": 127.0452,
        "distance": 320.0,
        "isUnlocked": false,
        "viewCount": 85
      }
    ],
    "pageInfo": {
      "currentPage": 0,
      "totalPages": 3,
      "totalElements": 45,
      "isLast": false
    }
  },
  "message": "해당 카테고리의 주변 둥지 목록을 성공적으로 조회했습니다."
}
```


엽서(Postcard) 등록 API

해금된 메시지에 사용자가 직접 사진과 텍스트를 담은 엽서를 작성하여 업로드하고 공유하기 위한 API이다.

- Endpoint: POST /api/v1/postcards
- Request Body:

```
{
  "nestId": 101,
  "title": "아주대 정문에서 본 노을",
  "content": "오늘 노을이 정말 예뻐서 한 장 남겨요. 다들 즐거운 하루 되시길!",
  "imageKey": "postcards/202604/uuid-filename.jpg"
}
```

- Response Body:

```
{
  "status": "SUCCESS",
  "data": {
    "postcardId": 501,
    "authorNickname": "한도훈",
    "imageUrl": "https://s3.ap-northeast-2.amazonaw
    "createdAt": "2026-04-01T17:40:00"
  },
  "message": "둥지에 엽서 등록이 완료되었습니다."
}
```

유저 프로필 및 FCM 토큰 저장 API

소셜 로그인 이후 유저의 닉네임, 선호 카테고리, 그리고 실시간 알림 전송을 위한 FCM 토큰 등 부가 정보를 저장 및 업데이트하는 API이다.

- Endpoint: PATCH /api/v1/users/me/profile
- Request Body:

```
{
  "nickname": "도도대장",
  "favoriteCategories": [1, 3, 5],
  "fcmToken": "fcm-token-string-abcd-1234"
  "profileImageUrl": "https://lh3.googleusercontent.com/..."
}
```

- Response Body:

```
{
  "status": "SUCCESS",
  "data": {
    "userId": 1,
    "nickname": "도도대장",
    "email": "dohun@gmail.com",
    "isProfileCompleted": true,
    "updatedAt": "2026-04-01T17:50:00"
  },
  "message": "유저 추가 정보 등록이 완료되었습니다. 이제 서비스를 정상적으로 이용할 수 있습니다."
}
```

5. 과제 수행 계획

5.1 일정계획

본 프로젝트의 세부 일정 및 진척도는 번다운 차트(Burndown Chart)로 실시간 관리하며, 본 제안서에서는 별도로 기재하지 않는다.

5.2 업무분장

기술적 구현을 위한 '개발 업무 분장'과 효율적인 팀 운영을 위한 '비개발 업무 분장'의 세부 사항을 기술한다. 단 번다운 차트에 명시된 것은 기술하지 않는다.

- 개발 업무 분장
  - Infrastructure & Backend : AWS 클라우드 환경 구축, CI/CD 파이프라인 설계 및 운영, DB 아키텍처 관리.
  - Frontend/Android Lead: 플랫폼별 코어 라이브러리 관리 및 UI/UX 일관성 유지 담당.
- 비개발 업무 분장
  - 협업 프로세스 총괄 (한도훈)
    - 플랫폼 세팅: GitHub Organization 구축 및 Discord 연동을 통한 실시간 알림 시스템 수립.
    - 워크플로우 규정: GitHub Issue/PR 템플릿 제작 및 Branch Protection Rule 설정을 통한 코드 품질 관리.
  - 디자인 및 UI/UX 설계 (최환희)
    - Figma 프로토타이핑: 서비스 전체 와이어프레임 및 고충실도(Hi-Fi) 디자인 제작.
    - 에셋 관리: 앱 내 사용되는 아이콘, 이미지 등 그래픽 리소스 가이드라인 수립.
  - 문서화 및 진척 관리 (정의훈)

5.3 코드 품질 향상을 위한 협업 방식

5.3.1 git branch 사용 방식

안정적인 메인 소스 관리와 병렬 개발을 위해 Git Flow 전략을 변형하여 적용한다.

- **main:** 제품으로 출시될 수 있는 상태의 코드가 관리되는 브랜치이다.
- **develop:** 다음 출시 버전을 대비하여 개발 중인 코드가 통합되는 브랜치이다.

- **feature(fix)/{issue-number}:** 기능 단위 개발 브랜치이다. 작업 완료 후 develop으로 PR을 수행한다.
- **hotfix:** 출시 버전에서 발생한 긴급 버그 수정 전용 브랜치이다.

5.3.2 DevOps 파이프라인 구성

코드 품질 유지와 배포 자동화를 위해 GitHub Actions와 Docker를 활용한 파이프라인을 구축한다.

- Backend
  - CI (Continuous Integration):
    - main, develop 브랜치에 Push/PR 발생 시 자동 빌드 테스트
    - 테스트 실패 시 Merge를 차단하여 코드 안정성 확보.
  - CD (Continuous Deployment):
    - 테스트 완료된 소스를 linux/amd64 아키텍처 기반 Docker 이미지로 빌드.
    - Docker Hub 푸시 후 AWS EC2에 자동으로 최신 이미지를 배포(Deploy) 및 docker compose 구동

프론트엔드는 Github Actions와 Vercel를 활용하여 빠른 빌드와 안정적인 배포 파이프라인을 구축한다.

- Frontend
  - CI
    - TypeScript 정적 분석을 통해 데이터 타입 불일치 및 런타임 에러 가능성 사전 차단.
    - next build를 수행하여 실제 운영 환경으로의 빌드 가능 여부를 최종 검증.
  - CD
    - CI 단계를 통과한 소스 코드를 바탕으로 Vercel 서버리스 인프라 최적화 빌드 수행.
    - 모든 PR 단계에서 실제 배포 전 UI/UX를 미리 확인할 수 있는 고유의 미리보기(Preview) URL을 자동 생성 및 제공.
    - main 브랜치 병합 시, 실서비스 도메인으로 최신 결과물을 자동 배포.

안드로이드는 Fast lane을 통해 배포를 한다.

5.3.3 테스트 자동화 방식

5.3.3.1 Backend

JUnit 5

- 역할: 자바 표준 테스트 프레임워크로서 테스트의 전체 생명주기 및 실행 관리를 담당한다.
- 상세 설명: @Test, @BeforeEach, @AfterEach 등의 어노테이션을 통해 테스트 케이스를 구조화하며, 현대적인 자바 에코시스템과의 높은 호환성을 제공한다.

AssertJ

- 역할: 가독성 높은 검증(Assertion) 라이브러리이다.
- 상세 설명: assertThat()과 같은 메서드 체이닝 기반의 문법을 제공하여 테스트 코드의 가독성을 높이고, 직관적인 에러 메시지를 통해 디버깅 효율을 향상시킨다.

Mockito

- 역할: 외부 의존성을 격리하는 모킹(Mocking) 프레임워크이다.
- 상세 설명: 데이터베이스나 외부 API 등 실제 객체를 사용하기 어려운 환경에서 가짜 객체(Mock)를 생성 및 관리하여, 단위 테스트의 독립성과 실행 속도를 보장한다.

MockMvc

- 역할: 컨트롤러 레이어 테스트를 위한 전용 도구이다.
- 상세 설명: 실제 서블릿 컨테이너(Tomcat 등)를 구동하지 않고도 HTTP 요청과 응답 규격을 검증할 수 있어, API 엔드포인트의 무결성을 신속하게 테스트한다.

GitHub Actions

- 역할: CI(지속적 통합) 파이프라인의 핵심 자동화 도구이다.
- 상세 설명: 모든 코드 변경 시점(Push/PR)마다 정의된 테스트 코드를 자동으로 실행하며, 테스트 미통과 시 병합(Merge)을 차단하여 코드 품질을 상향 평준화한다.

5.3.3.2 Frontend

Jest

- 역할: 비즈니스 로직 및 기능 단위 테스트(Unit Test) 수행한다.
- 상세 설명: UI 렌더링과 독립된 자바스크립트 함수 및 알고리즘의 정확성을 검증한다. 특히 Next.js의 핵심 비즈니스 로직이 포함된 Custom Hooks의 상태 변화와 반환값을 추적하여, 데이터 가공 및 상태 관리 로직의 신뢰성을 확보한다.

Storybook

- 역할: 컴포넌트 단위 고립 개발 및 상호작용 테스트(Interaction Test) 수행한다.
- 상세 설명: Play 함수를 활용해 사용자의 클릭, 타이핑 등 연속적인 동작을 스크립트로 자동화하며, 실제 브라우저 환경에서 UI 요소의 반응과 속성 변화를 정밀하게 검사한다.

Chromatic

- 역할: UI 디자인 일관성 및 레이아웃을 위한 테스트와 리뷰를 진행한다.
- 상세 설명: Storybook과 연동하여 모든 컴포넌트의 스냅샷을 관리하고 코드 변경 시 이전 버전과의 픽셀 단위 비교를 통해 의도치 않은 CSS 변경이나 레이아웃 깨짐을 자동으로 감지하며, 클라우드 환경에서 팀원 간 UI 변경 사항 승인 프로세스를 지원한다.

Playwright

- 역할: 서비스의 핵심 사용자 여정에 대한 E2E 테스트 자동화 및 브라우저 런타임 환경 검증을 수행한다.
- 상세 설명: '로그인 → 핀 선택 → 위치 인증 → 콘텐츠 해금'으로 이어지는 메인 시나리오를 자동 검증한다. 특히 안드로이드와 웹뷰 간의 데이터 연동 및 API 통신 상태를 실제 브라우저 수준에서 테스트한다.

Github Actions

- 역할: 지속적 통합(CI) 및 테스트 파이프라인 자동화
- 상세 설명: PR 발생 시 설정된 워크플로우를 자동으로 실행한다. 모든 테스트를 통과한 코드만 병합될 수 있도록 제어하여 안정성을 높인다.

5.3.3.3 Android

JUnit 5

- 역할: 안드로이드 로컬 단위 테스트(Unit Test)의 기본 프레임워크이다.
- 상세 설명: 안드로이드 프레임워크에 의존하지 않는 순수 코틀린 로직(ViewModel의 비즈니스 로직, 데이터 가공 알고리즘 등)을 JVM 환경에서 빠르게 검증한다. @Test와 같은 표준 어노테이션을 사용하여 테스트 수트의 구조를 관리한다.

MockK

- 역할: 코틀린 환경에 최적화된 모킹(Mocking) 라이브러리이다.
- 상세 설명: 자바 기반의 Mockito보다 코틀린의 suspend 함수(Coroutines)나 static 메서드 모킹에 특화되어 있다. Repository나 API 통신 등 외부 의존성을 가짜 객체로 대체하여 ViewModel이 의도한 대로 동작하는지 격리 테스트를 수행한다.

Compose UI Test (Semantics)

- 역할: Jetpack Compose 기반 UI 구성 요소의 선언적 검증 도구이다.
- 상세 설명: createComposeRule을 활용하여 화면에 특정 텍스트나 아이콘이 올바르게 렌더링되었는지 확인한다. 사용자의 클릭, 스크롤 등 UI 인터랙션을 시뮬레이션하고, 그 결과로 상태(State)가 변경되어 화면이 갱신되는지 노드 단위로 검증한다.

Espresso

- 역할: 안드로이드 네이티브 뷰 및 시스템 상호작용 테스트 도구이다.
- 상세 설명: 실제 기기나 에뮬레이터 환경에서 실행되는 계측 테스트(Instrumented Test)를 담당한다. 웹뷰(WebView) 내의 요소 선택이나 시스템 대화상자(권한 허용 등)와의 상호작용을 자동화하여, 하드웨어 기능과 연동된 시나리오의 무결성을 테스트한다.

GitHub Actions

- 역할: 안드로이드 빌드 및 테스트 자동화(CI) 파이프라인이다.
- 상세 설명: 새로운 코드 푸시(Push) 발생 시 점검용 에뮬레이터 인스턴스를 실행하거나 유닛 테스트를 자동 수행한다. 안드로이드 빌드 산출물(APK/AAB) 생성 전 모든 테스트 통과 여부를 확인하여, 기능 결함이 있는 버전이 배포되는 것을 사전에 차단한다.

5.3.4 코드 리뷰 방식

- PR(Pull Request) 생성 시, Gemini API, Github Copilot를 연동한 GitHub Actions를 통해 즉각적인 AI 리뷰를 수행한다.
- 매주 1회, main 브랜치로의 최종 병합(Merge) 직전에 전 팀원이 참여하는 대면 리뷰 세션을 진행한다.

5.3.5 코드 컨벤션

5.3.5.1 Git 컨벤션

커밋 메시지 형식 (Format)

type/#issueNumber: subject

예시: fix/#2: 보안 패치 완료

분류	태그 (Type)	내용 (Description)
기능/수정	feat	새로운 기능 추가
	fix	버그 수정
	!HOTFIX	**치명적인 버그를 급하게 수정해야 하는 경우**
품질/구조	refactor	코드 리팩토링 (기능 변화 없이 구조만 개선)
	style	코드 포맷팅, 세미콜론 누락 등 (코드 로직 변경 없음)
	test	테스트 코드 추가 및 리팩토링

기타 작업	docs	문서 수정 (README, Wiki, Swagger 등)
	design	CSS 등 사용자 UI/UX 디자인 변경
	chore	패키지 매니저, .gitignore 설정 등 그 외 모든 부수적인 작업

5.3.5.2 BE 컨벤션

→ JAVA convention을 따른다.

- **클래스명**: 대문자로 시작하는 파스칼 케이스(PascalCase)를 사용한다 (예: NestService, PostcardController).
- **메서드 및 변수명**: 소문자로 시작하는 카멜 케이스(camelCase)를 사용한다 (예: unlockNest(), viewCount).
- **상수명**: 모두 대문자를 사용하며 언더바(_)로 구분하는 스네이크 케이스(UPPER_SNAKE_CASE)를 사용한다 (예: MAX_UNLOCK_RADIUS).
- **패키지명**: 해당 패키지의 성격을 나타내는 단수형 명사를 사용하여 소문자로 작성한다 (예: com.dodo.domain.nest).

5.3.5.3 FE 컨벤션

→ JavaScript convention을 따른다.

- **컴포넌트명**: 모든 React 컴포넌트 파일은 파스칼 케이스(PascalCase)를 사용한다.
- **변수, 함수명**: 일반적인 변수, 상수, 함수 이름은 카멜 케이스(camelCase)를 사용한다.
- **상수명**: 환경변수나 변경되지 않는 설정값은 스네이크 케이스(UPPER_SNAKE_CASE)를 사용한다.
- **컴포넌트 선언**: 모든 컴포넌트는 함수 표현식(화살표 함수)를 이용하여 선언하다.
- **이벤트 핸들러:** Props로 전달되는 함수는 on 접두사를, 내부에서 처리하는 함수는 handle 접두사를 사용한다. (예: onDelete, handleDelete)
- **타입 정의:** 데이터 구조를 정의할 때는 interface 사용을 지향하며, 파일 상단에 위치시키거나 별도의 .types.ts 파일로 분리한다.

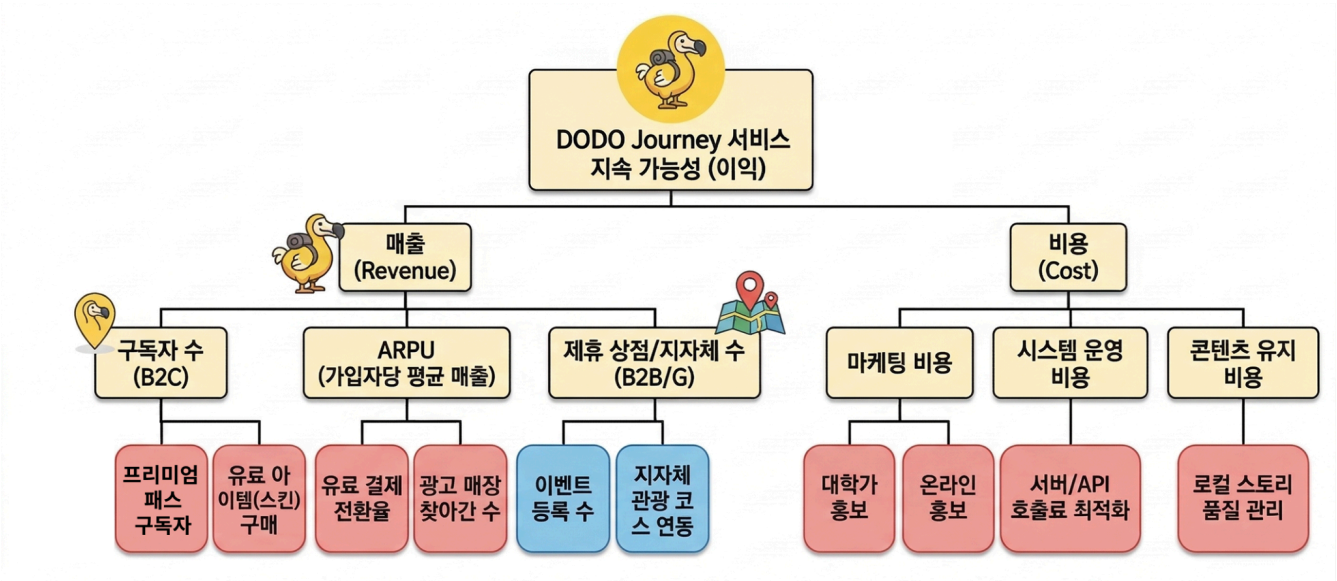
5.3.5.4 AN 컨벤션

→ Kotlin convention을 따른다.

- **클래스, 인터페이스명**: 대문자로 시작하는 파스칼 케이스(PascalCase)를 사용한다.
- **메서드 및 변수명**: 소문자로 시작하는 카멜 케이스(camelCase)를 사용한다.
- **상수명, 열거형**: 모두 대문자를 사용하며 언더바(_)로 구분하는 스네이크 케이스(UPPER_SNAKE_CASE)를 사용한다.
- **패키지명**: 해당 패키지의 성격을 나타내는 단수형 명사를 사용하여 소문자로 작성한다.
- **연산자**: 앞뒤에는 항상 공백을 추가한다.
- **중괄호**: K&R 스타일(Egyptian Brackets)을 따른다. 여는 중괄호는 줄 끝에, 닫는 중괄호는 새 줄에 작성한다.
- **콜론**: 타입의 경우 뒤에 공백 한칸, 상속&인터페이스의 경우 앞뒤로 공백 한칸 둔다.
- Unit 반환은 생략한다.
- 함수 본문이 한줄인 경우 = 을 사용하여 간결하게 표현한다.
  (ex. fun getSum(a: Int, b: Int) = a + b)
- 클래스는 프로퍼티, init 블록, 부 생성자, 메서드, 동반 객체 순으로 작성한다.

5.4 성과 측정 방안

5.4.1 비용적 KPI 트리



5.4.2 기능적 KPI 목록

5.4.2.1 [BE] KPI

백엔드 파트는 서버 인프라의 기술적 응답 성능과 배포 파이프라인의 안정성을 검증한다.

- 지표 1: API 응답 속도 (Latency)
  목표: 지도 데이터 및 이미지 메타데이터 호출 시 평균 200ms 이내 응답.
  측정: 서버 로그 및 모니터링 툴 활용.

- 지표 2: 시스템 가용성 및 배포 안정성
  목표: 목표: GitHub Actions 가동률 99% 인프라 장애 발생 시 MTTR 30분 이내 달성
  측정: GitHub Actions 실행 이력 및 Docker 컨테이너 상태 모니터링, 장애 복구 시간(MTTR)

기록.

5.4.2.2 [FE] KPI

프론트엔드 파트는 사용자가 콘텐츠(엽서, 게시물)에 얼마나 몰입하는지, 그리고 관리자가 시스템을 얼마나 효율적으로 통제하는지를 측정한다.

- 지표 1: 이탈률(Bounce Rate)
  목표: 웹뷰 페이지 평균 이탈률 40% 이하로 유지.
  측정: Vercel Analytics를 통한 이탈률 확인.

- 지표 2: 웹뷰 콘텐츠 로딩 성능 (LCP)
  목표: 주요 게시물 및 엽서 이미지의 최대 콘텐츠 페인팅(LCP) 시간을 2.5초 이내로 유지.
  측정: Vercel Speed Insights에서 제공하는 성능 지표 분석을 통한 사용자 체감 속도 확인

- 지표 3: 관리 효율성
  목표: 신고 접수 및 광고 승인 요청에 대한 평균 처리 시간(Mean Time To Resolve)을 24시간 이내로 유지.
  측정: 관리자 대시보드 내 '신고 처리 타임스탬프' 및 '광고 승인 로그' 데이터를 활용한 상시 모니터링.

5.4.2.3 [AN] KPI

사용자 경험 및 리텐션 (Retention) 안드로이드 파트는 사용자가 앱을 얼마나 쾌적하게 사용하고, 다시 방문하는지를 측정한다.

- 지표 1: 일일 활성 사용자 수 (DAU) 및 재방문율
  목표: 런칭 후 한 달 내 초기 타겟(대학생)의 20% 재방문 달성.
  측정: 구글 맵 탐험 횟수 및 엽서 확인 빈도 분석.

- 지표 2: 앱 성능 안정성 (Crash-free Users)
  목표: 비정상 종료 없는 사용자 비율 99% 이상 유지.
  측정: Firebase Crashlytics를 통한 실시간 에러 트래킹